

WEEK 6 SKILL OSSP

Part 1 — Backslash Escaping / Special Character Processing

1. Created a new folder for week 6 skill

```
ashborn@DESKTOP-I4G5Q4N:~$ mkdir week6_s
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/week6_s
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ pwd
/home/ashborn/week6_s
```

2. Basic Backslash Escaping

```
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo "Hello\nWorld"
Hello\nWorld
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo -e "Hello\nWorld"
Hello
World
```

3. Escaped Spaces

```
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo Hello\ World
Hello World
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo Hello World
Hello World
```

4. Escape Special Characters

```
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo \*
*
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo \&
&
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo \|
|
```

5. Preserve Characters

```
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo '$HOME'
$HOME
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo "$HOME"
/home/ashborn
```

6. Validate Parser Output

```
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ printf '<%s>\n' Hello\ World
<Hello World>
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ printf '<%s>\n' "Hello World"
<Hello World>
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ printf '<%s>\n' 'Hello World'
<Hello World>
```

7. Test Complex Input

```
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ echo "Hello\ World \$USER \& \*"
Hello\ World $USER \& \*
```

Summary:

The experiment demonstrates how backslash escaping is used in Linux shell processing to preserve spaces and special characters. Escape sequences such as `\n` were tested, along with escaped spaces, shell metacharacters, quoted strings, and complex inputs.

The outputs were validated using `printf` to observe how the shell parses individual arguments. The experiment confirms that escaping and quoting can control how the shell interprets special characters and preserves intended input.

PART 2 — Child Processes

1. Creating the Child Process Program, Compiling and Execute

```

ashborn@DESKTOP-I4G5Q4N:~/week6_s$ nano child_process.c
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ gcc child_process.c -o child_process
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ ./child_process
Parent Process: PID = 34
Child Process: PID = 35
Waiting for child process...
Executing command: ls
total 36
-rwxr-xr-x 1 ashborn ashborn 16264 Sep 17 17:29 child_process
-rw-r--r-- 1 ashborn ashborn  640 Sep 17 17:29 child_process.c
Child process completed.

```

2. Passing Arguments to a Child Process

```

ashborn@DESKTOP-I4G5Q4N:~/week6_s$ nano child_process.c
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ gcc child_process.c -o child_process
ashborn@DESKTOP-I4G5Q4N:~/week6_s$ ./child_process
Parent Process: PID = 42
Child Process: PID = 43
Running: echo Hello OSSP Week 6
Hello OSSP Week 6
Child process completed.

```

3. Final Process Check

```

ashborn@DESKTOP-I4G5Q4N:~/week6_s$ ps
  PID TTY          TIME CMD
   11 tty1      00:00:00 bash
   52 tty1      00:00:00 ps

```

Summary:

The experiment demonstrated backslash escaping and special character processing in Ubuntu. We tested escaped spaces, special symbols, escape sequences, character preservation using quotes, and validated how the shell parses normal and complex inputs using printf.

We also implemented process creation and execution in C using fork(), exec(), and wait(). The child process executed commands with arguments, the parent process managed the child, and execution errors were handled using perror().

